The parentheses around the arguments may be omitted if the expression is otherwise unambiguous.

Positional arguments are assigned to the matching parameters of the method. Following these arguments may be a list of _key_: _value_ pairs, which correspond to keyword parameters of the method.

Any argument may be a prefixed with an asterisk. If a starred argument supports the to_a method, that method is called, and the resulting array is expanded inline to provide arguments to the method call. If a starred argument does not support to_a, it is simply passed through unaltered.

```
def regular(a, b, *c)
  "a = #{a}, b = #{b}, c = #{c}"
end
regular 1, 2, 3, 4                    # => a = 1, b = 2, c = [3, 4]
regular(1, 2, 3, 4)                   # => a = 1, b = 2, c = [3, 4]
regular(1, *[2, 3, 4])                # => a = 1, b = 2, c = [3, 4]
regular(1, *[2, 3], 4)                # => a = 1, b = 2, c = [3, 4]
regular(1, *[2, 3], *4)               # => a = 1, b = 2, c = [3, 4]
regular(*[], 1, *[], *[2, 3], *[], 4) # => a = 1, b = 2, c = [3, 4]
```

Any argument may be prefixed with two asterisks (a double splat). Such arguments are treated as hashes, using to_hash to convert if the parameter is not a hash, and their key-value pairs are added as additional keyword arguments to the method call.

```
def regular(a, b)
  "a = #{a}, b = #{b}"
end
regular(99, a: 1, b: 2)                       # => a = 99, b = {:a=>1, :b=>2}

others = { c: 3, d: 4 }
regular(99, a: 1, b: 2, **others)             # => a = 99, b = {:a=>1, :b=>2,
                                              # .. :c=>3, :d=>4}
regular(99, **others, a: 1, b: 2)             # => a = 99, b = {:c=>3, :d=>4,
                                              # .. :a=>1, :b=>2}

rest = {e: 5}

regular(99, **others, a: 1, b: 2)             # => a = 99, b = {:c=>3, :d=>4,
                                              # .. :a=>1, :b=>2}
regular(99, **others, a: 1, b: 2, **rest)     # => a = 99, b = {:c=>3, :d=>4,
                                              # .. :a=>1, :b=>2, :e=>5}
```

When a method defined with keyword parameters is called, Ruby matches the keys in the passed hash with each parameter, assigning values when it finds a match.

```
def keywords(a, b: 2, c: 3)
  "a = #{a}, b = #{b}, c = #{c}"
end

keywords(99)                        # => a = 99, b = 2, c = 3
keywords(99, c: 98)                 # => a = 99, b = 2, c = 98

args = {b: 22, c: 33}
keywords(99, **args)                # => "a = 99, b = 22, c = 33"
keywords(99, **args, b: "override") # => "a = 99, b = override, c = 33"
```